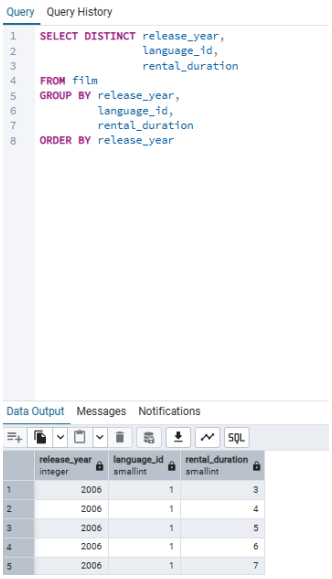
3.6: Summarizing & Cleaning Data in SQL

Ben Diemler

1- Check and clean dirty data for customer table and film table

- **Non-uniform**

Film table:



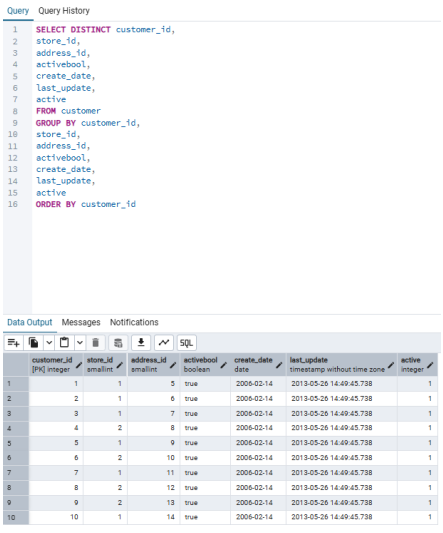
```
SELECT DISTINCT release_year,
                language_id,
                rental_duration
FROM film
GROUP BY release_year,
         language_id,
         rental_duration
ORDER BY release_year
```

Data Output | Messages | Notifications

	release_year integer	language_id smallint	rental_duration smallint
1	2006	1	3
2	2006	1	4
3	2006	1	5
4	2006	1	6
5	2006	1	7

```
SELECT DISTINCT release_year,
        language_id,
        rental_duration
FROM film
GROUP BY release_year,
    language_id,
    rental_duration
ORDER BY release_year
```

Customer table:



```
SELECT DISTINCT customer_id,
store_id,
address_id,
activebool,
create_date,
last_update,
active
FROM customer
GROUP BY customer_id,
store_id,
address_id,
activebool,
create_date,
last_update,
active
ORDER BY customer_id
```

Data Output | Messages | Notifications

	customer_id [PK] integer	store_id smallint	address_id smallint	activebool boolean	create_date date	last_update timestamp without time zone	active integer
1	1	1	5	true	2006-02-14	2013-05-26 14:49:45.738	1
2	2	1	6	true	2006-02-14	2013-05-26 14:49:45.738	1
3	3	1	7	true	2006-02-14	2013-05-26 14:49:45.738	1
4	4	2	8	true	2006-02-14	2013-05-26 14:49:45.738	1
5	5	1	9	true	2006-02-14	2013-05-26 14:49:45.738	1
6	6	2	10	true	2006-02-14	2013-05-26 14:49:45.738	1
7	7	1	11	true	2006-02-14	2013-05-26 14:49:45.738	1
8	8	2	12	true	2006-02-14	2013-05-26 14:49:45.738	1
9	9	2	13	true	2006-02-14	2013-05-26 14:49:45.738	1
10	10	1	14	true	2006-02-14	2013-05-26 14:49:45.738	1

```
SELECT DISTINCT customer_id,
store_id,
address_id,
activebool,
create_date,
last_update,
active
FROM customer
GROUP BY customer_id,
store_id,
address_id,
activebool,
create_date,
last_update,
active
ORDER BY customer_id
```

I focused the initial inconsistency analysis on structured fields with fixed data types. These columns were prioritized because their inherent format (numerical IDs, date standards, and true/false) simplifies the automated detection of deviations and errors. On the other hand, free-form text fields, such as first and last names, movie titles or emails, were excluded from this phase, as differentiating simple variations from genuine, systemic inconsistencies is significantly more challenging to spot.

There were no inconsistencies found on either table. However, if the results were otherwise, I would clean this data by spotting what exactly needs to be fixed, and update the values accordingly to match the others. For example, if on customer table I found active bool as ‘T’, ‘True’, ‘F’, ‘False’ results, I would then proceed with the following query:

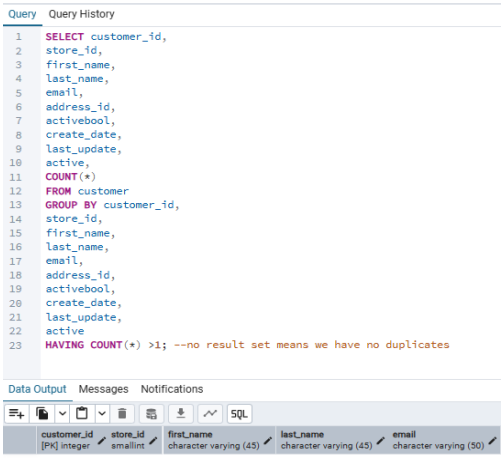
```
UPDATE customer
SET activebool = 'True'
WHERE rating IN ('T');

UPDATE customer
SET activebool = 'False'
WHERE rating IN ('F');
```

- **Duplicate data**

Film table:


```
Query:
SELECT title,
    release_year,
    language_id,
    rental_duration,
    COUNT(*)
FROM film
GROUP BY title,
     release_year,
     language_id,
     rental_duration
HAVING COUNT(*) >1; --no result set means we have no duplicates
```

Customer table:




```
SELECT customer_id,
store_id,
first_name,
last_name,
email,
address_id,
activebool,
create_date,
last_update,
active,
COUNT(*)
FROM customer
GROUP BY customer_id,
store_id,
first_name,
last_name,
email,
address_id,
activebool,
create_date,
last_update,
active
HAVING COUNT(*) >1; --no result set means we have no duplicates
```

Obs: As there were no instructions regarding which columns to select, I decided to select all columns from the table to get actual full duplicates. However, if I wanted, for example, to find duplicate cst accounts, I would most likely stop on address_id, as these would be personal customer information, and not account activity data.

The results from the query show that there are no duplicates on both tables. However, if results were different, and there was the need to clean duplicate data, I would follow the standard fix method, which is create a virtual table (view), where only unique records would be selected.

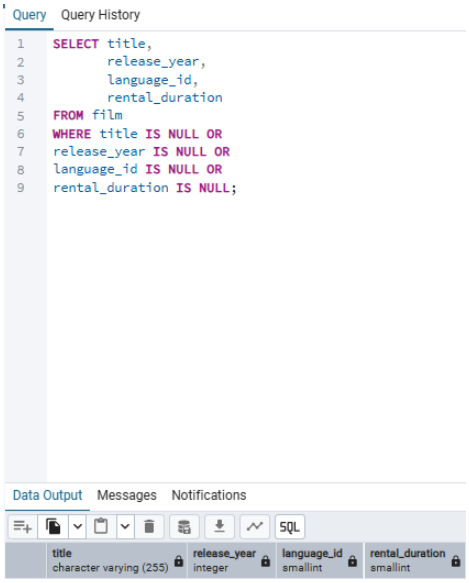
To create this view with unique records, I would use the following query:

```
Film table:
CREATE VIEW viewname AS
SELECT title,
    release_year,
    language_id,
    rental_duration
FROM film
GROUP BY title,
    release_year,
    language_id,
    rental_duration --Group by will
make each row unique
```

```
Customer table:
CREATE VIEW viewname AS
SELECT customer_id,
store_id,
first_name,
last_name,
email,
address_id,
activebool,
create_date,
last_update,
active,
FROM customer
GROUP BY customer_id,
store_id,
first_name,
last_name,
email,
address_id,
activebool,
create_date,
last_update,
active --Group by will make each row unique
```

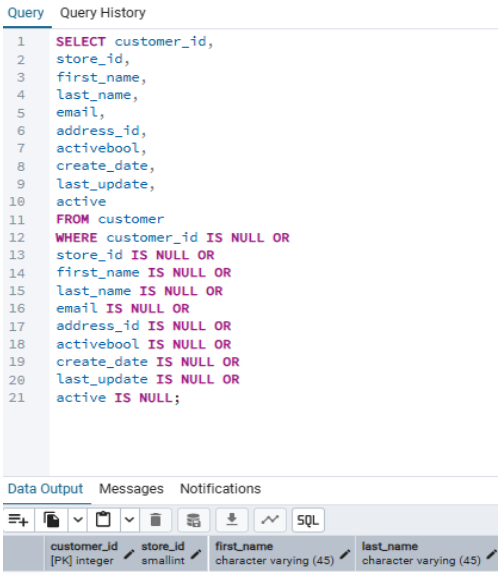
- **Missing values**

Film table:




```
SELECT title,
    release_year,
    language_id,
    rental_duration
FROM film
WHERE title IS NULL OR
release_year IS NULL OR
language_id IS NULL OR
rental_duration IS NULL;
```

Customer table:




```
SELECT customer_id,
store_id,
first_name,
last_name,
email,
address_id,
activebool,
create_date,
last_update,
active
FROM customer
WHERE customer_id IS NULL OR
store_id IS NULL OR
first_name IS NULL OR
last_name IS NULL OR
email IS NULL OR
address_id IS NULL OR
activebool IS NULL OR
create_date IS NULL OR
last_update IS NULL OR
active IS NULL;
```

For this exercise, I actually considered using the OR function that we previously learned, combined with the NULL value showed on the missing values example query, to input missing values. In this way, I would be able to find any NULL row on the columns selected for each table.

As you can see, there were no null results received. However, if data cleaning was needed, I would take one of the following two approaches:

- If there were really few null values, I would input missing values with average, using the following query for example:

```
--imputing missing values with the AVG value
UPDATE customer
SET = AVG(last_updated)
WHERE last_updated IS NULL
```

- If there were too many values in the column, I would just ignore it on my select statement to avoid compromising my analysis, for example:

```
SELECT title,
    Release_year,
    Rental_duration –language_id ignored in
select because it has a lot of missing values
FROM Film
```

2- Summarize data:

Non-numerical query (used for each relevant group):

```
SELECT MODE () WITHIN GROUP (ORDER BY title)
AS modal_title
FROM film;
```

Numerical query (used for each relevant group):

```
SELECT MIN (release_year) AS min_release_year,
    MAX (release_year) AS max_release_year,
            AVG (release_year)AS avg_release_year,
            COUNT(release_year) AS count_release_year,
            COUNT(*) AS count_rows
FROM film;
```

Film table outputs:

Category	MIN	MAX	AVG	Count	Count_rows	Mode
Film_id	1	1000	500.5	1000	1000	
title						Academy Dinosaur
description						A Action-Packed Character Study of a Astronaut And a Explorer who must Reach a Monkey in A MySQL Convention
Release_year	2006	2006	2006	1000	1000	
Language_id						1
Rental_duration	3	7	4.985	1000	1000	
Rental_rate	0.99	4.99	2.98	1000	1000	
Length	46	185	115.27	1000	1000	
Replacement_cost	9.99	29.99	19.98	1000	1000	
Rating						PG-13
Last_update	2013-05-26 14:50:58.951	2013-05-26 14:50:58.951		1000	1000	
Special_features						{Trailers,Commentaries,"Behind the Scenes"}
fulltext						'baloon':19 'confront':14 'documentari':5 'feminist':8,11,16 'mile':2 'must':13 'spi':1 'thrill':4

Customer table:

Category	MIN	MAX	AVG	Count	Count_rows	Mode
Customer_id	1	599	300	599	599	
Store_id						1
First_name						Jamie
Last_name						Abney
Email						aaron.selby@sakilacustomer.org
Address_id						5
Activebool						true
Create_date	2006-02-14	2006-02-14		599	599	
Last_update	2013-05-26 14:49:45.738	2013-05-16 14:49:45.738		599	599	
Active						1

Obs: Although active, language_id, address_id and store_id are number results, I did not consider it as numeric as they are codes that represent certain information (ex: 1= English, 2 = Mandarin). In this way, if I make an average, I could get 1.5, which would not be a relatable code, which is why as added mode instead.

3- Reflecting on work

For data profiling, I prioritize SQL due to its superior language structure and speed when managing volume. The more proficient I become with complex queries, the faster my analysis runs, which is critical for large datasets. I reserve Excel for smaller data volumes, where its simplicity and familiar interface provide quick, lightweight solutions.